

A User-Centred, From-Scratch Scientific Calculator for the Tangent Function $\tan(x)$

Deliverable 1: Persona, Requirements, Algorithms, CLI Prototype

Risat Ahmed
Student ID: 40294116

SOEN 6011 – Software Engineering Processes, Section CC
Concordia University – Summer 2026

July 15, 2026

Outline

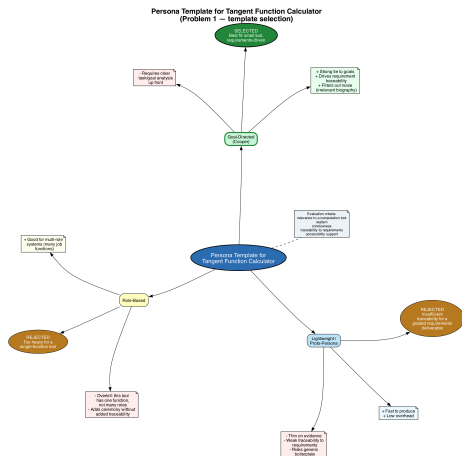
- 1 Persona
- 2 Requirements
- 3 Algorithms
- 4 Selection & CLI
- 5 Verification & Limitations
- 6 GAI Prompts
- 7 References

Problem 1 — Persona: Template Selection

Three template styles compared:
goal-directed, role-based,
lightweight/proto-persona.

Selected: goal-directed — ties every
persona field to a later requirement,
matching a single-function calculator's
design.

Role-based rejected: too heavy for one
function. Lightweight rejected: too thin
on traceability.



Problem 1 — Persona: Maya Chen

- **Role:** M.A.Sc. Mechanical Engineering student, optics-lab research assistant
- **Goal:** a quick, trustworthy $\tan(x)$ value without opening a full CAS
- **Pain points:** radians/degrees confusion; distrust of results near $\pi/2$; no patience for raw tracebacks

7 traceable needs identified (gate: ≥ 5) $\rightarrow$ directly drive the Problem 2 requirements.

Problem 2 — Requirements (ISO/IEC/IEEE 29148)

18 requirements, categories: FR VAL ACC USE REL ERR DOC. Each has an ID, statement, persona-traced rationale, priority, verification method.

ID	Statement	Pri.
VAL-03	Reject input whose $ x $ exceeds the documented maximum (10^4 rad); report the input as out of the supported range.	Must
VAL-04	Detect input x within a documented tolerance of a pole ($x = \pi/2 + k\pi$) and treat it as an invalid computation rather than returning a numeric value.	Must
ACC-01	For accepted x outside the pole-tolerance band and within the supported range, compute $\tan(x)$ with relative error $\leq 10^{-9}$ vs. trusted reference values.	Must
ERR-01	When input is rejected under any VAL requirement, display a message identifying the specific condition violated.	Must

Quality review caught 3 issues: an unquantified term ("fast and accurate"), an algorithm prescribed too early, a non-singular requirement – all corrected before freezing **Baseline v0.1**.

Problem 3 — Algorithm A: Range Reduction + Maclaurin Series

```
k <- ROUND(x / (PI/2))
r <- x - k*(PI/2) // r in [-pi/4,pi/4]
sinR <- MACLAURIN-SIN(r)
cosR <- MACLAURIN-COS(r)
if |cosR| < EPSILON
  signal POLE-ERROR
return sinR / cosR
```

Convergence: super-linear, ~ 10 – 15 terms for 10^{-12}

Trace: $\tan(\pi/4) = 1$ confirmed

Weakness: reduction error grows with $|x|$

Problem 3 — Algorithm B: CORDIC (Circular, Rotation Mode)

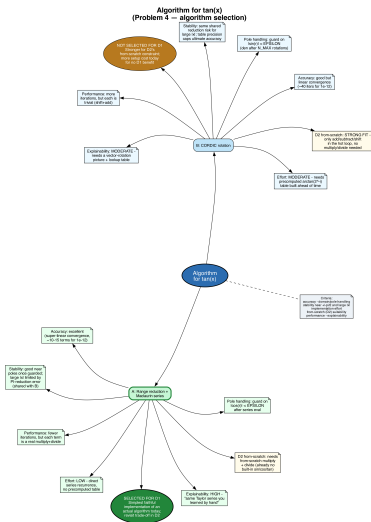
```
vx <- 1 ; vy <- 0 ; z <- r
for i <- 0 to N_MAX - 1
  if z >= 0 : d <- +1 else d <- -1
  vx_next <- vx - d*vy*2^(-i)
  vy_next <- vy + d*vx*2^(-i)
  z <- z - d*A[i] // A[i]=arctan(2^-i)
  vx <- vx_next ; vy <- vy_next
if |vx| < EPSILON
  signal POLE-ERROR
return vy / vx // K cancels for tan
```

Convergence: linear, ~ 1
bit/iteration, ~ 40 iterations for
 10^{-12}

Trace: $\tan(\pi/4) = 1$ confirmed
(0.986 after 7 iters $\rightarrow$
1.000000000000 after 39)

Weakness: finite table precision
caps ultimate accuracy

Problem 4 — Algorithm Selection



Selected: Algorithm A for the D1 prototype.

Lowest implementation effort, most explainable to the persona.

Algorithm B (CORDIC) noted as the stronger candidate for D2's from-scratch, multiplication-free constraint – revisit then.

Problem 4 — CLI Prototype (src/cli.py)

- Compute layer (maclaurin_sin/cos, tan_by_series) separated from I/O layer
- Two custom exceptions: `OutOfRangeError`, `NearPoleError`
- States angle unit + domain hint before first prompt; 10 significant figures displayed

Live Demo / Sample Runs

```
terminal -- python3 src/cli.py (near-pole guard)

tan(x) calculator (Deliverable 1 prototype)
- Enter x as a real number, in RADIANS (not degrees).
- Supported range:  $|x| \leq 10000$  radians.
- Values at or within tolerance of a pole ( $x = \pi/2 + k\pi$ ) are rejected
  with an explanation rather than returning a huge/meaningless number.
- Results are displayed to 10 significant figures.
- Type 'quit' or 'exit' to leave.

Enter x (radians), or 'quit' to exit: Cannot compute tan(x): x is at or within tolerance of
  a pole ( $x = \pi/2 + k\pi$ ), where  $\cos(x)$  is approximately zero and  $\tan(x)$  is undefined.
Enter x (radians), or 'quit' to exit: Cannot compute tan(x): x is at or within tolerance of
  a pole ( $x = \pi/2 + k\pi$ ), where  $\cos(x)$  is approximately zero and  $\tan(x)$  is undefined.
Enter x (radians), or 'quit' to exit: Goodbye.
```

Verification Highlights

Case	Result	Verdict
$x = \pi/4$	$\tan = 1$	Pass
$x = -\pi/4$	$\tan = -1$ (odd symmetry)	Pass
$x = \pi/2 - 10^{-9}$	rejected (near-pole)	Pass
$x = 1000$	1.470324156, matches <code>math.tan</code>	Pass
$x = 10^{10}$	rejected (out of range)	Pass

All demo cases traced to a requirement ID; full matrix in `docs/verification_matrix.md`.

Known Limitations

- Large-argument accuracy degrades gradually below $|x| \leq 10^4$; exact threshold not yet quantified
- Pole tolerance (10^{-6}) and X_MAX (10^4) are provisional, pending professor confirmation
- No performance/timing requirement verified (deliberately deferred)
- Reference values rely on `math.tan` (finite precision), adequate here but not an arbitrary-precision oracle

GAI Prompt — Problem 1 (Persona)

Tool: Claude Code (Sonnet 5) **Type:** Generative + decision-support

TASK Compare ≥ 3 persona-template styles (goal-directed, role-based, lightweight/proto) for a $\tan(x)$ calculator user as a mind-map outline; recommend one; then draft one persona (name, role, experience, skills, goals, environment, tan-specific needs/pain points) + usage scenario.

RESTRICTIONS No invented statistics or real-person claims; every attribute must trace to ≥ 5 later requirements; no implementation technology prescribed.

OUTPUT FORMAT (1) template-comparison mind map + decision; (2) persona card + assumptions + 3–5 sentence usage scenario.

AUDIENCE SOEN 6011 professor/TAs/grad SE students – evidence-based, traceable to requirements.

GAI Prompt — Problem 2 (Requirements)

Tool: Claude Code (Sonnet 5) **Type:** Generative + transformational, with review

TASK Express requirements per ISO/IEC/IEEE 29148 style, informed by the finalized Problem 1 persona; ID scheme FR/VAL/ACC/USE/REL/PERF/ERR/DOC; cover accepting x , angle unit, computing/displaying $\tan(x)$, clear/retry/exit, rejecting invalid/near-pole/extreme inputs, a measurable accuracy tolerance; list assumptions separately; review the draft against the 29148 quality characteristics.

RESTRICTIONS No algorithm/library/language prescribed inside a requirement; no vague unquantified terms; each requirement singular and independently verifiable.

OUTPUT FORMAT Requirements table (ID, Statement, Rationale/persona trace, Priority, Verification) + Assumptions list + Review-findings table.

AUDIENCE Same – standard-aligned, testable, traceable.

GAI Prompt — Problem 3 (Two Algorithms)

Tool: Claude Code (Sonnet 5) **Type:** Generative + comparative

TASK Specify two independent, language-neutral algorithms in established pseudocode for $\tan(x)$, satisfying pasted Problem 2 requirement IDs (ACC-01, VAL-03, VAL-04, REL-01). Algorithm A: range reduction (period π , odd symmetry) + Maclaurin series for $\sin / \cos$. Algorithm B: CORDIC rotation (precomputed $\arctan(2^{-i})$ table). For each: inputs/outputs/pre-postconditions/constants/validation/stopping condition/max-work safeguard/complexity/weaknesses/worked trace.

RESTRICTIONS No real programming-language syntax; every loop must terminate or have a max-work cap; no built-in $\sin / \cos / \tan$ assumed; algorithms must be genuinely different.

OUTPUT FORMAT Two pseudocode listings, each with pre/post note, complexity/limitations note, worked trace; "How A and B differ" comparison table.

AUDIENCE Same – rigorous, unambiguous, independently implementable.

GAI Prompt — Problem 4 (Selection + CLI)

Tool: Claude Code (Sonnet 5) **Type:** Decision-support + generative

TASK Step 1: mind-map comparing the two Problem 3 algorithms on accuracy, domain/pole handling, stability, effort, D2 from-scratch suitability, performance, explainability; select one, justify rejecting the other. Step 2: Python 3 CLI implementing the selected algorithm; separate I/O from compute; prompt for x + angle unit + domain hint; validate; detect near-pole; compute; display with documented precision; catch invalid/numerical-failure cases.

RESTRICTIONS Selection must follow the stated criteria and anticipate the D2 from-scratch constraint; must run from a terminal with no unhandled traceback on invalid/near-pole input; compute core independently callable; may use Python's `math` in D1, but note which calls need from-scratch replacement in D2.

OUTPUT FORMAT (1) selection mind map + decision; (2) full Python source; (3) sample runs (valid/near-pole/negative/invalid) + command used.

AUDIENCE Same – evidence-based selection, clean/robust code, runnable outside an IDE.

- A. Cooper et al., *About Face 3*. Wiley, 2007.
- J. Pruitt, T. Adlin, *The Persona Lifecycle*. Morgan Kaufmann, 2006.
- ISO/IEC/IEEE 29148:2018, Requirements Engineering.
- J. E. Volder, "The CORDIC Trigonometric Computing Technique," *IRE Trans. Electronic Computers*, 1959.
- J.-M. Muller et al., *Handbook of Floating-Point Arithmetic*, 2nd ed., 2016.
- T. Cormen et al., *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.
- NIST Digital Library of Mathematical Functions, <https://dlmf.nist.gov/>.

Thank You

Questions?